COMP 1752 Object Oriented Programming

For your reference

sample

University of Greenwich
D-No 001400809

Contents

Contents 2
Executive Summary 3
1. Introduction 4
2. Diagrams 4
Class diagram 4
2.1 Brief explanation of the class diagram 4
Stage 1 5
3. Commenting 5
Stage 2 6
Outline design 6
Email manager Outline design 6
Read message Outline design 6
New Message Outline design 7
Label message outline design 7
Stage 3 8
Email manager coding 8
Read message code 10
Label message code 11
New message code 12
Stage 4 13
4.1. Testing 13
5. Screenshots of the Program working 15
a. Screen 1 Email manager 15
Brief description 15
Screen 2- Read message 15
Brief description 16
Screen 3 Label messages 16
Brief description 16
Screen 4 New messages 17
Brief description 17
Evaluation 18

What went well? 18
What went less well? 18
What I learned? 18
AI declaration Form 19

Executive Summary

I have asked to produce a simulation of an email manager.

Firstly, I have introduced the coursework and started working on the diagrams. I made class diagram and gave a brief explanation after that I have designed use case diagram.

Then I started my Stage 1 which was commenting on the code.

Stage 2 is the outline design of the simulation of the email manager, read message and new message with code written in a prototype.

Stage 3 is about inserting the code for email manager, read message, label message and new message as well as the output.

Stage 4 is the testing of the GUI. Testing for each function in the email manager simulation is helpful to identify any bugs present possible.

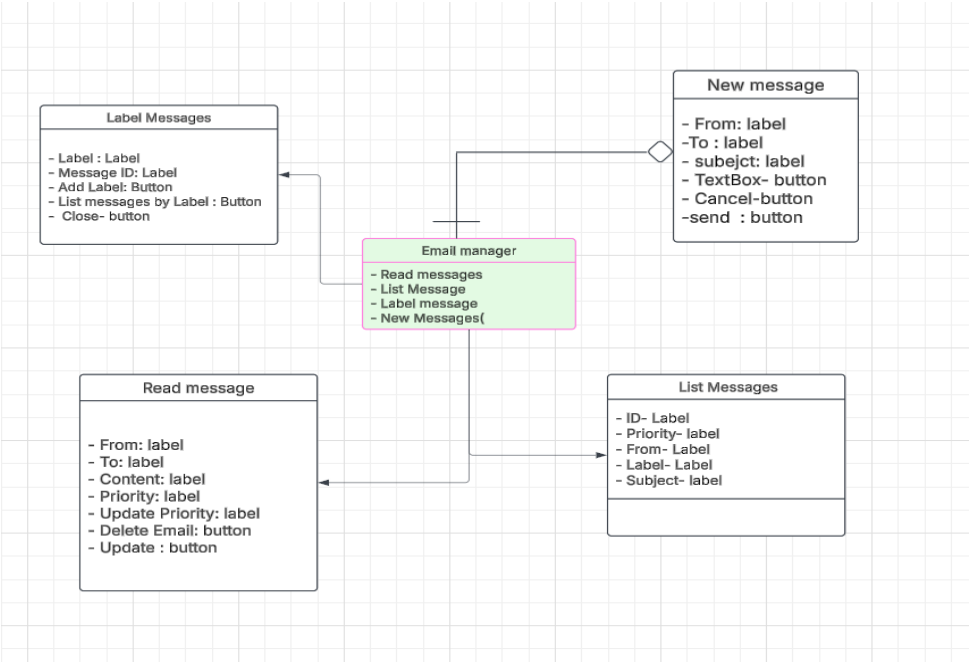
Lastly, I have evaluated about all over coursework and discussed about what went well, what went less well and what I learned. This coursework was helpful for enhancing my knowledge in coding field and how to work with it more effectively.

1. Introduction

I was asked to produce a simulation of an email manager system. The goal is to create an email manager a simple version of Outlook. I have used VS code to work with this coursework. Functionality involving Read message, label message, new message, and list message. By doing this coursework I can gain knowledge about how to create real time application, improving my technical programming skills. This coursework main goal is to understand software development practise, particularly the python. The simulation of the Email manager should work effectively by the end of the course work. At the end of the course, it will improve my coding skills and making functional communication platform.

2. Diagrams

Class diagram



2.1 Brief explanation of the class diagram

Email manager is the main controller for handling email interfaces. Contain actions of reading emails, List messages, Label messages, new messages. New massages will be received. Read massages will be focused on reading messages. Email represents an individual email that contains the sender ID, subject, recipient, and the timestamp. This class diagram shows how the email simulation will work. How manageable and organized the data and the methods will be.

Stage 1

3. Commenting

C: > Users > islam > OneDrive > UofG Year 1 > Comp 1752 object oriented programing > read_message stage 1 Commenting.py > ...

```python
import tkinter as tk # Import the tkinter library, which is used for creating graphical user interfaces (GUIs).
import tkinter.scrolledtext as tkst # Import the scrolledtext module from tkinter, providing a text widget with a scroll bar.

import message_manager as messages # type: ignore
import font_manager as fonts # type: ignore


class ReadMessage(): # Define a new class named ReadMessage, which represents a message-reading window.
    def __init__(self, window, message_id):  # Define the constructor method, initializing the class with parameters for the window and message ID.
        self.message_id = message_id # Store the provided message ID as an instance variable to reference later.

        self.window = window # Store the provided window (a tkinter window object) as an instance variable.
        self.window.geometry("500x320")  # Set the dimensions of the window to 500 pixels wide and 320 pixels tall.
        self.window.title(f"Read Message {message_id}") # Set the title of the window dynamically, including the message ID.

        sender_lbl = tk.Label(window, text="From:") # Create a label widget with the text "From:" and associate it with the specified window
        sender_lbl.grid(row=0, column=0, sticky="E", padx=10, pady=10)
        # Place the label in the first row (row 0), first column (column 0) of the grid layout, aligning it to the east (right) and adding padding of 10 pixels on both x and y axes.


        self.sender_txt = tk.Entry(window, width=40) # Create a text entry widget (single line) for the sender's information, setting its width to 40 characters.
        self.sender_txt.grid(row=0, column=1, columnspan=5, sticky="W", padx=10, pady=10)
        # Place the entry widget in the first row (row 0), starting from the second column (column 1). It spans 5 columns and is aligned to the west (left) with padding.

        recipient_lbl = tk.Label(window, text="To:") # Create a label widget with the text "To:" and associate it with the specified window.
        recipient_lbl.grid(row=1, column=0, sticky="E", padx=10, pady=10)
        # Place the label in the second row (row 1), first column (column 0) of the grid layout, aligning it to the east (right) and adding padding of 10 pixels on both x and y axes.

        self.recipient_txt = tk.Entry(window, width=40)  # Create a text entry widget for inputting the recipient's information, setting its width to 40 characters.
        self.recipient_txt.grid(row=1, column=1, columnspan=5, sticky="W", padx=10, pady=10)
        # Place the entry widget in the second row (row 1), starting from the second column (column 1). It spans 5 columns, aligns to the west (left), and includes padding of 10 pixels

        subject_lbl = tk.Label(window, text="Subject:")  # Create a label widget with the text "Subject:" and associate it with the specified window.
        subject_lbl.grid(row=2, column=0, sticky="E", padx=10, pady=10)
        # Position the label in the third row (row 2), first column (column 0), aligning it to the east (right) with padding of 10 pixels

        self.subject_txt = tk.Entry(window, width=40) # allowing users to input a single line of text
        self.subject_txt.grid(row=2, column=1, columnspan=5, sticky="W", padx=10, pady=10)
        # Places the `subject_txt` Entry widget in the grid layout at row 2 and column 1, spanning 5 columns, left-aligned (sticky="W") with 10px padding on all sides (padx=10, pady=10

        self.content_txt = tkst.ScrolledText(window, width=48, height=6, wrap="word")
        # Creates a ScrolledText widget, a multi-line text input field with a scrollbar, inside the `window` parent container. The text box has a width of 48 characters, a height of 6
```

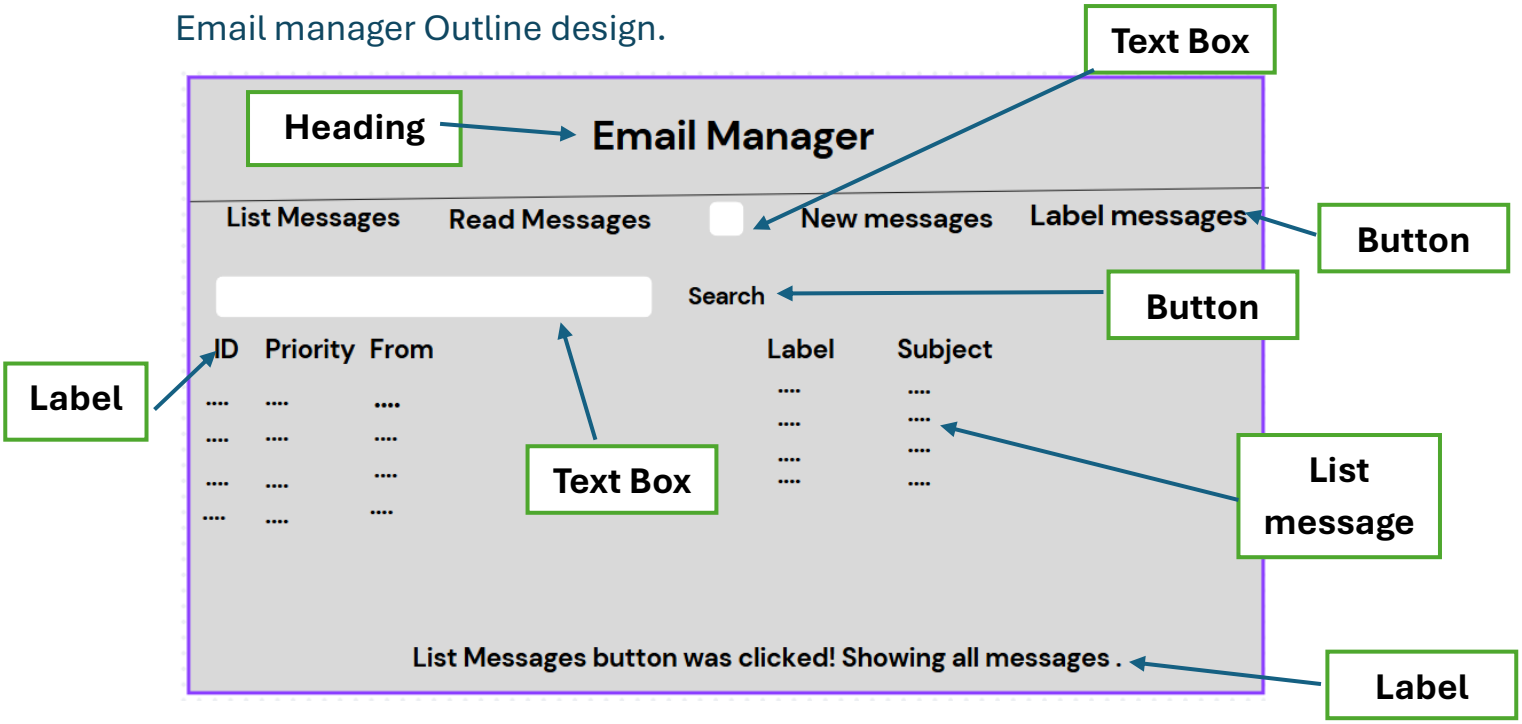
```python
        self.content_txt.grid(row=3, column=0, columnspan=6, sticky="W", padx=10, pady=10)
        # Positions the ScrolledText widget in the grid layout at row 3 and column 0. It spans across 6 columns, is aligned to the left (sticky="W"), and has 10 pixels of padding both horizontally (padx) and vertically (pady).


        subject_lbl = tk.Label(window, text="New priority (1-5):") #this line creates a label in the GUI that reads "New priority (1-5):", probably to provide context for some user input.
        subject_lbl.grid(row=4, column=0, columnspan=2, sticky="E", padx=10, pady=10) # this line positions and aligns the subject_lbl label while ensuring it's well-spaced within the layout

        self.priority_txt = tk.Entry(window, width=3) #, this line creates a small input box within the specified window, where the user can type, likely for entering a priority value.
        self.priority_txt.grid(row=4, column=2, sticky="W", padx=10, pady=10) #This line effectively positions the Entry field next to the label created earlier, providing a space for user input with proper alignment and spacing in

        update_btn = tk.Button(window, text="Update", command=self.update_priority) # This line creates a Button widget using the tkinter library
        update_btn.grid(row=4, column=3, sticky="W", padx=10, pady=10) # This line places the button within the grid layout of the parent container (`window`).

        delete_btn = tk.Button(window, text="Delete", command=self.delete_message) # Creates a "Delete" button in the specified parent container and links it to the `delete_message` method.
        delete_btn.grid(row=4, column=4, padx=10, pady=10) # Positions the button in grid row 4, column 4, with 10 pixels of padding around it.

        close_btn = tk.Button(window, text="Close", command=self.close)
Creates a Button widget labeled "Close" within the `window` container and binds its functionality to the `self.close` method.
        close_btn.grid(row=4, column=5, padx=10, pady=10) #Places the close_btn button on a grid at row 4, column 5, with horizontal (padx) and vertical (pady) padding of 10 pixels.

        if message_id is not None: # Checks if the variable message_id has a valid value (not None).
            sender = messages.get_sender(message_id)# Checks if the variable message_id has a valid value (not None).
            if sender is not None:# Ensures that the sender variable contains valid information before proceeding.
                self.sender_txt.insert(tk.END, sender) #Inserts the sender information at the end of the sender_txt text widget
                self.sender_txt.configure(state='readonly') # Sets the sender_txt text widget to "readonly" mode, preventing further edits.
                self.recipient_txt.insert(tk.END, messages.get_recipient(message_id)) # Inserts the recipient information (retrieved using get_recipient(message_id)) at the end of the recipient_txt text widget.
                self.recipient_txt.configure(state='readonly') # Sets the recipient_txt text widget to "readonly" mode, preventing further edits.
                self.subject_txt.insert(tk.END, messages.get_subject(message_id)) # Inserts the subject information (retrieved using get_subject(message_id)) at the end of the subject_txt text widget.
                self.subject_txt.configure(state='readonly') # Sets the subject_txt text widget to "readonly" mode, preventing further edits.
                self.content_txt.insert(tk.END, messages.get_content(message_id)) # Inserts the message content (retrieved using get_content(message_id)) at the end of the content_txt text widget.
            else: # Marks the start of the code block to handle the case when sender is None.
                self.content_txt.insert(tk.END, 'No such message') # Displays "No such message" in the content_txt text widget when no message content is available.
        self.content_txt["state"] = "disabled" # Disables the content_txt text widget to prevent user edits.

    def delete_message(self): # Defines the delete_message method to handle message deletion.
        if self.message_id is not None: # Checks if self.message_id has a valid value before attempting to delete the message.
            messages.delete_message(self.message_id) # Deletes the message corresponding to the self.message_id using the delete_message method.
        self.close() # Calls the close method to close the current window after the message is deleted.

    def update_priority(self): # Defines the update_priority method to change the message's priority.
        if self.message_id is not None: # Checks if self.message_id is valid before updating the priority.
            messages.set_priority(self.message_id, int(self.priority_txt.get())) # Updates the message's priority based on the value entered in the priority_txt widget.

    def close(self): # Defines the close method to handle closing the application window.
        self.window.destroy() # Destroys the application window, effectively closing it.


f __name__ == "__main__":  # only runs when this file is run as a standalone
    window = tk.Tk()  # create a TK object
    fonts.configure()  # configure the fonts
    ReadMessage(window, None)  # open the ReadMessage GUI
    window.mainloop()  # run the window main loop, reacting to button presses, etc
```

Figure 1Commenting about the message code.

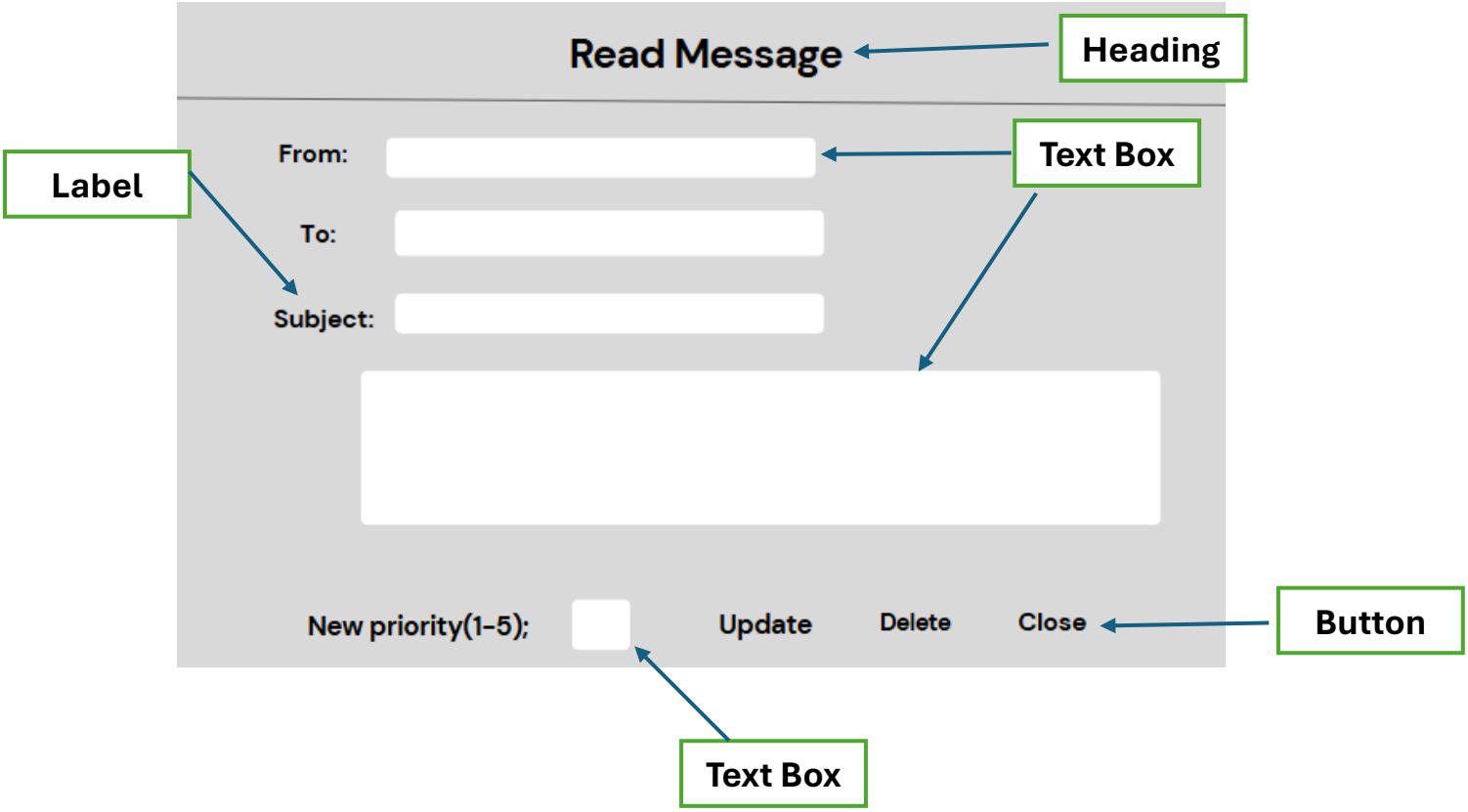
Stage 2

Outline design

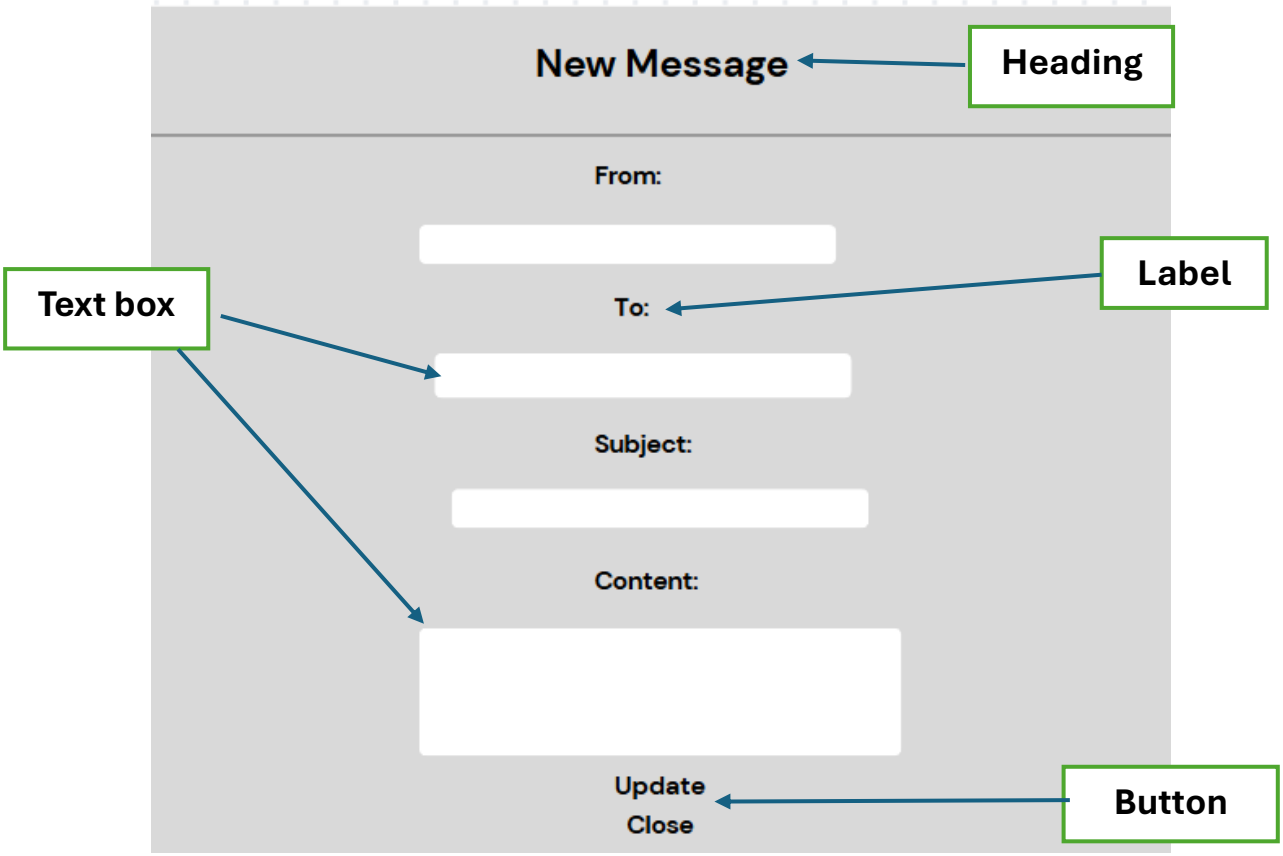
Email manager Outline design.



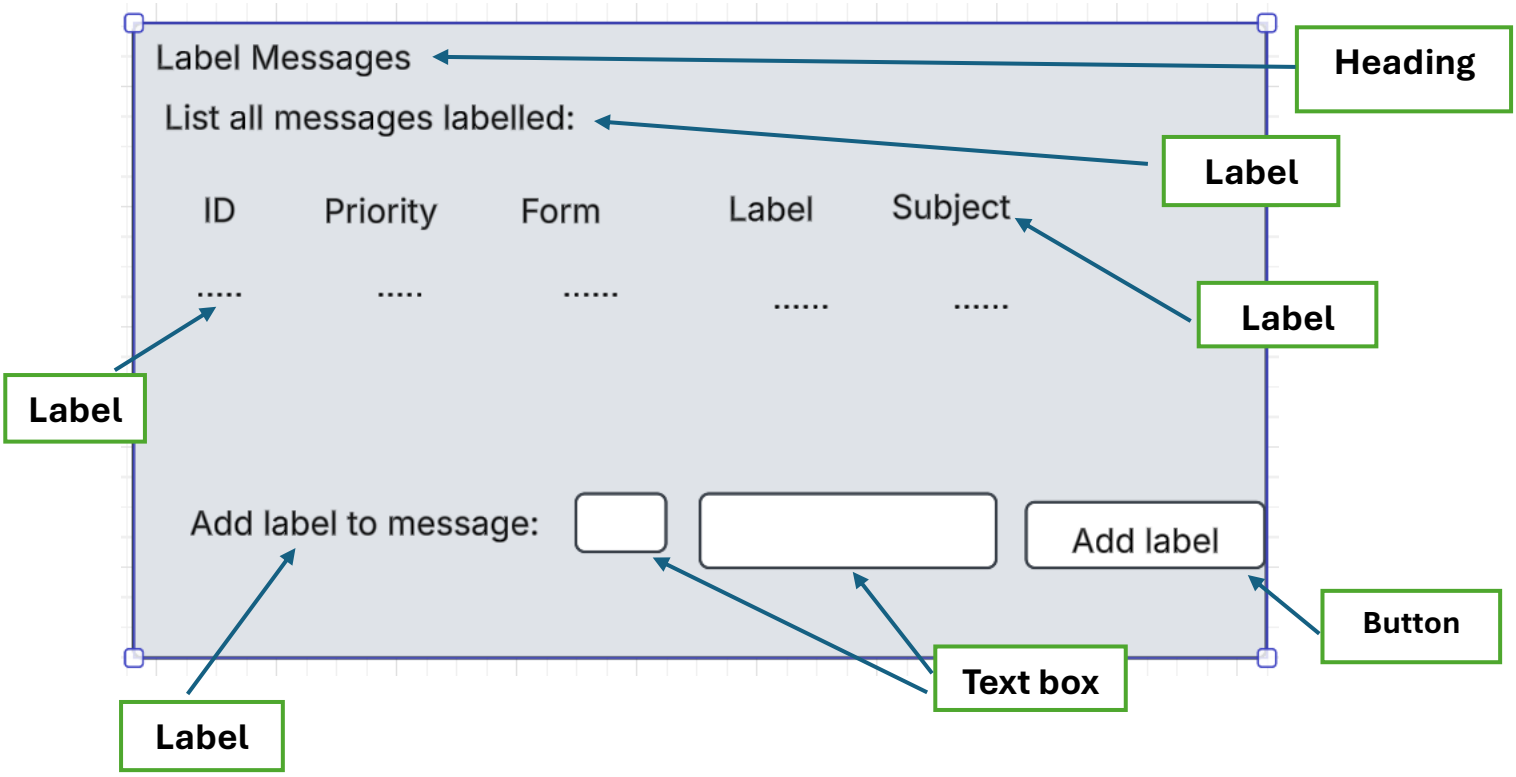
Read message Outline design.



New Message Outline design



Label message outline design.



Stage 3

Email manager coding.

```python
import tkinter as tk
from tkinter import messagebox
from new_message import NewMessage
from label_messages import LabelMessages
from message_manager import list_all, get_sender, get_recipient, get_subject, get_content, get_priority, set_priority, delete_message, messages

class EmailManager:
    def __init__(self, root):
        self.root = root
        self.root.title("Email Manager")
        self.root.geometry("800x600")  # Increased window size to accommodate email details

        # List Messages Button
        self.list_button = tk.Button(root, text="List Messages", command=self.list_messages)
        self.list_button.pack(pady=10)

        # New Message Button
        self.new_message_button = tk.Button(root, text="New Message", command=self.open_new_message)
        self.new_message_button.pack(pady=10)

        # Label Messages Button
        self.label_button = tk.Button(root, text="Label Messages", command=self.open_label_messages)
        self.label_button.pack(pady=10)

        # Text Area to Display Emails
        self.text_area = tk.Text(root, height=15, width=90)
        self.text_area.pack(pady=10)

        # Message ID Field for Reading Emails
        self.message_id_label = tk.Label(root, text="Enter Message ID to Read:")
        self.message_id_label.pack(pady=5)
        self.message_id_entry = tk.Entry(root)
        self.message_id_entry.pack(pady=5)

        # Read Message Button
        self.read_button = tk.Button(root, text="Read Message", command=self.read_message)
        self.read_button.pack(pady=10)

        # Email Details Frame
        self.details_frame = tk.Frame(root)
        self.details_frame.pack(pady=10)

        # Labels for Email Details
        self.sender_label = tk.Label(self.details_frame, text="From:")
```

```python
        self.sender_label.grid(row=0, column=0, sticky="w")
        self.sender_value = tk.Label(self.details_frame, text="")
        self.sender_value.grid(row=0, column=1, sticky="w")

        self.recipient_label = tk.Label(self.details_frame, text="To:")
        self.recipient_label.grid(row=1, column=0, sticky="w")
        self.recipient_value = tk.Label(self.details_frame, text="")
        self.recipient_value.grid(row=1, column=1, sticky="w")

        self.subject_label = tk.Label(self.details_frame, text="Subject:")
        self.subject_label.grid(row=2, column=0, sticky="w")
        self.subject_value = tk.Label(self.details_frame, text="")
        self.subject_value.grid(row=2, column=1, sticky="w")

        self.content_label = tk.Label(self.details_frame, text="Content:")
        self.content_label.grid(row=3, column=0, sticky="w")
        self.content_value = tk.Label(self.details_frame, text="", wraplength=600, justify="left")
        self.content_value.grid(row=3, column=1, sticky="w")

        self.priority_label = tk.Label(self.details_frame, text="Priority:")
        self.priority_label.grid(row=4, column=0, sticky="w")
        self.priority_value = tk.Label(self.details_frame, text="")
        self.priority_value.grid(row=4, column=1, sticky="w")

        # Update Priority Field and Button
        self.update_priority_label = tk.Label(self.details_frame, text="Update Priority:")
        self.update_priority_label.grid(row=5, column=0, sticky="w")
        self.update_priority_entry = tk.Entry(self.details_frame)
        self.update_priority_entry.grid(row=5, column=1, sticky="w")
        self.update_priority_button = tk.Button(self.details_frame, text="Update", command=self.update_priority)
        self.update_priority_button.grid(row=5, column=2, sticky="w")

        # Delete Button
        self.delete_button = tk.Button(self.details_frame, text="Delete Email", command=self.delete_email)
        self.delete_button.grid(row=6, column=0, columnspan=3, pady=10)

    def list_messages(self):
        """List all emails in the text area."""
        self.text_area.delete(1.0, tk.END)
        self.text_area.insert(tk.END, list_all())

    def open_new_message(self):
```

```python
    def open_new_message(self):
        """Open the New Message GUI."""
        NewMessage(self.root)

    def open_label_messages(self):
        """Open the Label Messages GUI."""
        LabelMessages(self.root)

    def read_message(self):
        """Display the details of the selected email in the main window."""
        try:
            message_id = int(self.message_id_entry.get())  # Get the Message ID from the entry field
            if message_id in messages:  # Check if the Message ID exists in the messages dictionary
                # Display email details
                self.sender_value.config(text=get_sender(message_id))
                self.recipient_value.config(text=get_recipient(message_id))
                self.subject_value.config(text=get_subject(message_id))
                self.content_value.config(text=get_content(message_id))
                self.priority_value.config(text=get_priority(message_id))
            else:
                messagebox.showerror("Error", "Invalid Message ID.")  # Show error if ID is invalid
        except ValueError:
            messagebox.showerror("Error", "Message ID must be a number.")  # Show error if input is not a number

    def update_priority(self):
        """Update the priority of the selected email."""
        try:
            message_id = int(self.message_id_entry.get())
            new_priority = int(self.update_priority_entry.get())
            if message_id in messages:
                set_priority(message_id, new_priority)
                self.priority_value.config(text=new_priority)  # Update the displayed priority
                messagebox.showinfo("Success", "Priority updated successfully.")
            else:
                messagebox.showerror("Error", "Invalid Message ID.")
        except ValueError:
            messagebox.showerror("Error", "Priority must be a number.")

    def delete_email(self):
        """Delete the selected email."""
        try:
            message_id = int(self.message_id_entry.get())
```

```python
            if message_id in messages:
                delete_message(message_id)
                self.clear_email_details()  # Clear the displayed email details
                messagebox.showinfo("Success", "Email deleted successfully.")
            else:
                messagebox.showerror("Error", "Invalid Message ID.")
        except ValueError:
            messagebox.showerror("Error", "Message ID must be a number.")

    def clear_email_details(self):
        """Clear the displayed email details."""
        self.sender_value.config(text="")
        self.recipient_value.config(text="")
        self.subject_value.config(text="")
        self.content_value.config(text="")
        self.priority_value.config(text="")
        self.update_priority_entry.delete(0, tk.END)

if __name__ == "__main__":
    root = tk.Tk()
    app = EmailManager(root)
    root.mainloop()
```

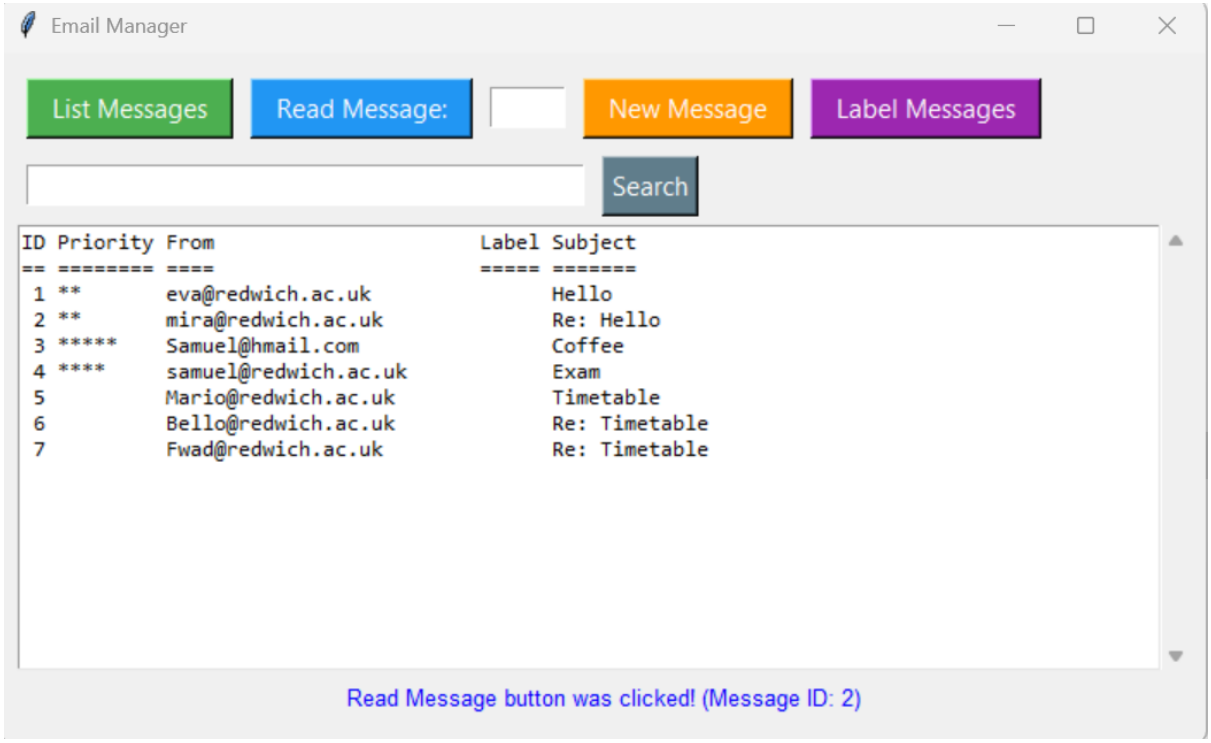


Figure 2 Outcome of the email manager code.

Read message code.

```python
#tkinker (a GUI development library) is imported ,the library is named for use of programing:
import tkinter as tk
#"scrolledtext" is imported from the tkinker library:
import tkinter.scrolledtext as tkst
#local classes imported for use:
import message_manager as messages
import font_manager as fonts

# The class "ReadMessage" is declared:
class ReadMessage():

    #object constructor .When on object "ReadMessage" is made the following occurs:
    def __init__(self, window, message_id):
        self.message_id = message_id
# The window variable is declared as window
        self.window = window
        #tkinter is used to resize the window:
        self.window.geometry("500x320")
        self.window.title(f"Read Message {message_id}")

#tkinter is used to create a label with the text "From":
        sender_lbl = tk.Label(window, text="From:")
        sender_lbl.grid(row=0, column=0, sticky="E", padx=10, pady=10)
# A variable called "Sender_txt" is assigned a tkinter text entery field in the "window":
        self.sender_txt = tk.Entry(window, width=40)
# The text entery field "sender_txt"is positioned in row 0 column 1. The text entery field is to set to occupy 5 columns.It is padded from the left:
        self.sender_txt.grid(row=0, column=1, columnspan=5, sticky="W", padx=10, pady=10)
# A variable calle "recipient_lbl" is declared and is assigned a tkinker label.This label is placed on the window the label has a text "To":
        recipient_lbl = tk.Label(window, text="To:")
# recipient_lbl is positioned on row 1 column 0 on the grid and is padded by 10 pixels iin the X and Y axis from the right side
        recipient_lbl.grid(row=1, column=0, sticky="E", padx=10, pady=10)
# A variable called "self.recipient_txt"(belongs to the object) is assigned a tkinker text entery field in "window" with a width of 40 pixels:
        self.recipient_txt = tk.Entry(window, width=40)
 #The text entery field "recipient_txt"is positioned in row 1 column 1. The text entery field is set so occupy 5 columns. It is padded from the left
        self.recipient_txt.grid(row=1, column=1, columnspan=5, sticky="W", padx=10, pady=10)
# A variable called "subject_lbl" is assigned a tkinker in "window" with text "subject" on it:
        subject_lbl = tk.Label(window, text="Subject:")
# The label "subject_lbl" is positioned on row 2 column 0 .It is padded on the right side by 10 pixels in the X and Y axis:
        subject_lbl.grid(row=2, column=0, sticky="E", padx=10, pady=10)
# # A variable called "subject_txt" is assigned a tkinker text entery is field in "window" with width 40 pixels
        self.subject_txt = tk.Entry(window, width=40)

        self.subject_txt.grid(row=2, column=1, columnspan=5, sticky="W", padx=10, pady=10)
# A variable called "content_txt" is assigned a tkinker scrolled text entery is field in "window" with width 48 pixels
```

```python
        self.content_txt = tkst.ScrolledText(window, width=48, height=6, wrap="word")
#A text entery field (content_txt) is possitioned on row 3 column 0 the text field entery is set to occupy 6 columns It is padded from the left
        self.content_txt.grid(row=3, column=0, columnspan=6, sticky="W", padx=10, pady=10)
# A variable called "subject_lbl" is assigned a tkinker in "window" with text "New priority (1-5)" on it:
        subject_lbl = tk.Label(window, text="New priority (1-5):")
# The label "subject_lbl" is positioned on row 4 column 0 .It is padded on the right side by 10 pixels in the X and Y axis:
        subject_lbl.grid(row=4, column=0, columnspan=2, sticky="E", padx=10, pady=10)
# A variable called "priority_txt" is assigned a tkinker scrolled text entery is field in "window" with width 3 pixels
        self.priority_txt = tk.Entry(window, width=3)
# the "priority_txt"is placed on row 4 column 2 it is padded from the left side by 10 pixels
        self.priority_txt.grid(row=4, column=2, sticky="W", padx=10, pady=10)
# A variable called "updated_btn" is assigned a tkinker button with the text "update"on it.Once clicked the "update_priority" method is
        update_btn = tk.Button(window, text="Update", command=self.update_priority)
#"update_btn" is placed on row 4 column 3 and is padded from the left side by 10 pixels
        update_btn.grid(row=4, column=3, sticky="W", padx=10, pady=10)
#A variable called "delete_btn" is assigned a tkinker button with the text "Delete"on it. the function of the variable is to delete message
        delete_btn = tk.Button(window, text="Delete", command=self.delete_message)
        delete_btn.grid(row=4, column=4, padx=10, pady=10)
#A variable called "close_btn" is assigned a tkinker button with the text "Close"on it.The function of this variable is to close the GUI
        close_btn = tk.Button(window, text="Close", command=self.close)
        close_btn.grid(row=4, column=5, padx=10, pady=10)
#Check is "message_id " is a null value .if it is not null the variable sender will be assigned using the method message.get_sender:
        if message_id is not None:
            sender = messages.get_sender(message_id)
# Check if "sender"is not a null value.if the value is null the else statement is executed:###########
            if sender is not None:
                self.sender_txt.insert(tk.END, sender)
#configures the text entery box "sender_txt" to a read only state
                self.sender_txt.configure(state='readonly')
#configures the text entery box "recipient_txt" to a read only state
                self.recipient_txt.insert(tk.END, messages.get_recipient(message_id))
                self.recipient_txt.configure(state='readonly')
#configures the text entery box "subject_txt" to a read only state
                self.subject_txt.insert(tk.END, messages.get_subject(message_id))
                self.subject_txt.configure(state='readonly')
#configures the text entery box "content_txt" to a read only state
                self.content_txt.insert(tk.END, messages.get_content(message_id))

        #executed if the above if statement is not met:
            else:
#The text"no such message" is output to the "content_txt" text box:
                self.content_txt.insert(tk.END, 'No such message')
```

```python
        self.content_txt["state"] = "disabled"
#A function called"delete_message" is created and providing message_id is not none the message is deleted
    def delete_message(self):
        if self.message_id is not None:
            messages.delete_message(self.message_id)
        self.close()
# A function called "update_priority" is defined and if message_id is not none the priority is met,and the message_id and the priority is set
    def update_priority(self):
        if self.message_id is not None:
            messages.set_priority(self.message_id, int(self.priority_txt.get()))




    def close(self):
        self.window.destroy()


if __name__ == "__main__":  # only runs when this file is run as a standalone
    window = tk.Tk()  # create a TK object
    fonts.configure()  # configure the fonts
    ReadMessage(window, None)  # open the ReadMessage GUI
    window.mainloop()  # run the window main loop, reacting to button presses, etc
```

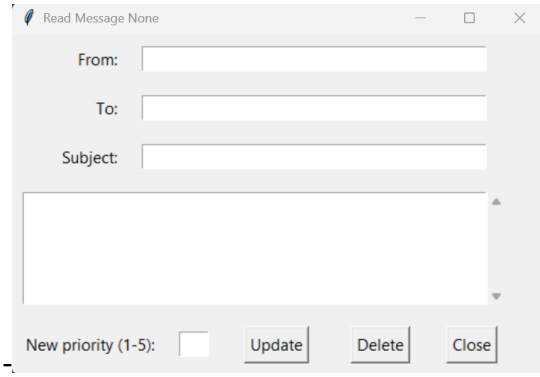


Figure 3 Outcome of the Read message code.

Label message code.

```python
import tkinter as tk
from tkinter import messagebox
from message_manager import list_all, set_label

class LabelMessages:
    def __init__(self, root):
        self.root = root
        self.window = tk.Toplevel(root)
        self.window.title("Label Messages")
        self.window.geometry("400x300")

        # Label Field
        self.label_label = tk.Label(self.window, text="Label:")
        self.label_label.pack(pady=5)
        self.label_entry = tk.Entry(self.window)
        self.label_entry.pack(pady=5)

        # Message ID Field
        self.message_id_label = tk.Label(self.window, text="Message ID:")
        self.message_id_label.pack(pady=5)
        self.message_id_entry = tk.Entry(self.window)
        self.message_id_entry.pack(pady=5)

        # Add Label Button
        self.add_label_button = tk.Button(self.window, text="Add Label", command=self.add_label)
        self.add_label_button.pack(pady=10)

        # List Messages by Label Button
        self.list_label_button = tk.Button(self.window, text="List Messages by Label", command=self.list_messages_by_label)
        self.list_label_button.pack(pady=10)

        # Text Area to Display Emails
        self.text_area = tk.Text(self.window, height=10, width=40)
        self.text_area.pack(pady=10)

        # Close Button
        self.close_button = tk.Button(self.window, text="Close", command=self.window.destroy)
        self.close_button.pack(pady=10)

    def add_label(self):
        try:
            message_id = int(self.message_id_entry.get())
            label = self.label_entry.get()
            if label:
```

```python
                set_label(message_id, label)
                messagebox.showinfo("Success", "Label added successfully.")
            else:
                messagebox.showerror("Error", "Label cannot be empty.")
        except ValueError:
            messagebox.showerror("Error", "Message ID must be a number.")

    def list_messages_by_label(self):
        label = self.label_entry.get()
        if label:
            self.text_area.delete(1.0, tk.END)
            self.text_area.insert(tk.END, list_all(label=label))
        else:
            messagebox.showerror("Error", "Label cannot be empty.")
```

Figure 4 Output of the label message code.

New message code

```python
import tkinter as tk
from tkinter import messagebox
from message_manager import new_message

class NewMessage:
    def __init__(self, root):
        self.root = root
        self.window = tk.Toplevel(root)
        self.window.title("New Message")
        self.window.geometry("400x300")

        # Sender Field
        self.sender_label = tk.Label(self.window, text="From:")
        self.sender_label.pack(pady=5)
        self.sender_entry = tk.Entry(self.window)
        self.sender_entry.pack(pady=5)

        # Recipient Field
        self.recipient_label = tk.Label(self.window, text="To:")
        self.recipient_label.pack(pady=5)
        self.recipient_entry = tk.Entry(self.window)
        self.recipient_entry.pack(pady=5)

        # Subject Field
        self.subject_label = tk.Label(self.window, text="Subject:")
        self.subject_label.pack(pady=5)
        self.subject_entry = tk.Entry(self.window)
        self.subject_entry.pack(pady=5)

        # Content Field
        self.content_label = tk.Label(self.window, text="Content:")
        self.content_label.pack(pady=5)
        self.content_entry = tk.Text(self.window, height=5, width=30)
        self.content_entry.pack(pady=5)

        # Send Button
        self.send_button = tk.Button(self.window, text="Send", command=self.send_email)
        self.send_button.pack(pady=10)

        # Cancel Button
        self.cancel_button = tk.Button(self.window, text="Cancel", command=self.window.destroy)
        self.cancel_button.pack(pady=10)

    def send_email(self):
```

```python
        sender = self.sender_entry.get()
        recipient = self.recipient_entry.get()
        subject = self.subject_entry.get()
        content = self.content_entry.get("1.0", tk.END)

        if sender and recipient and subject and content:
            new_message(sender, recipient, subject, content)
            messagebox.showinfo("Success", "Email sent successfully.")
            self.window.destroy()
        else:
            messagebox.showerror("Error", "All fields are required.")
```

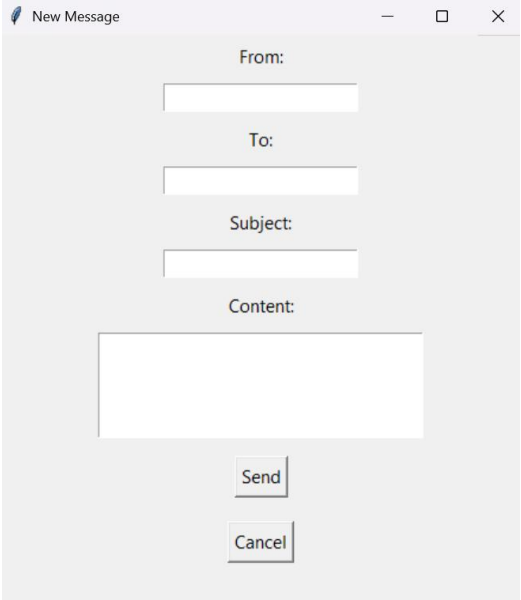
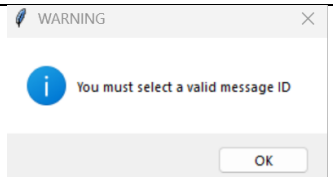


Figure 5 Outcome of the New message code.

Stage 4

4.1. Testing

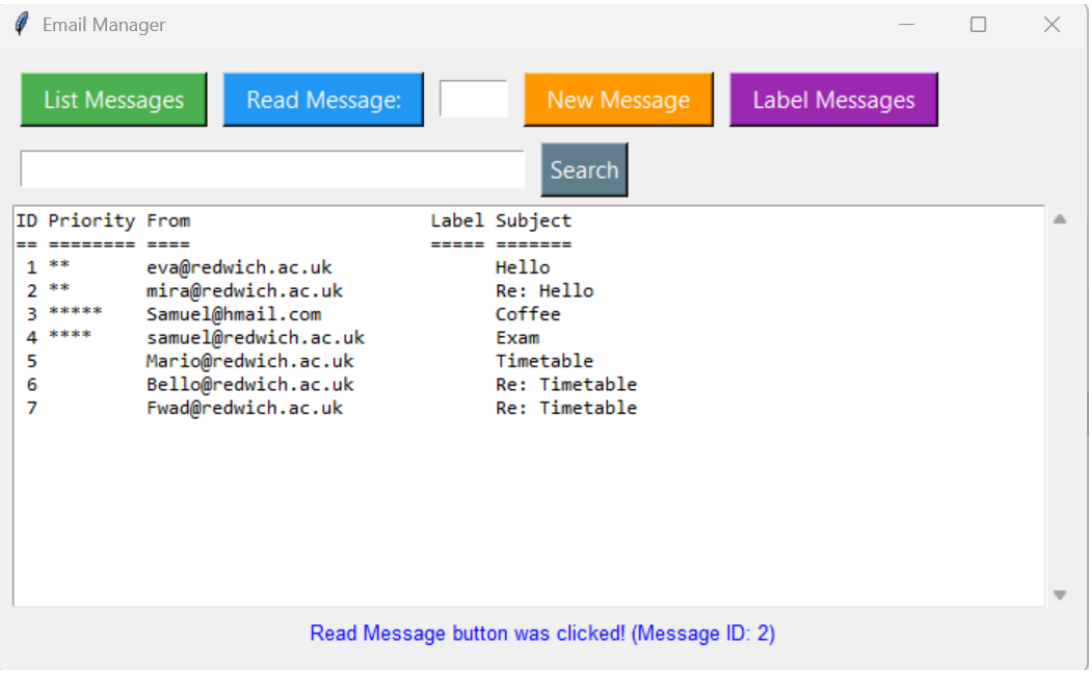
Testing for all classes

Test Number	Test Description	Expected Outcome	Actual Outcome	Pass/Fail
1	Email manager	When I run the email manger it should show me the email manager page.	It does show me the email manager page when I run it	Pass
2	Email manager	When I click the Read message button it should display another screen.	It does display another screen to read the messages	Pass
3	Email manager	When I click the new message button it should show me new message display to add new message.	It does display new message screen to add new messages.	Pass
4	Email manager	When I click the list messages button it should show me the list of messages.	It does display list of messages when the button is clicked in the email manager	Pass
5	Email manager	When I click the button for the label messages it shows me the label message screen.	It does display me the label message screen.	Pass
6	Read message	When I click the button "read message" in the email manager page. Without inputting any ID, it should show me an error.		Pass
7	Read Message	When I click the button "read message" in the email manager page. Inputting the ID, it should show me the information about that ID owner.	It does open the information of the ID owner	Pass
8	Read message	When the update button is clicked it should update the information.	It does update the information when the Update button is clicked.	Pass

9	Read message	When the Delete button is closed it should Delete the information that has been displayed.	It does delete the information when the Delete button is clicked.	Pass
10	Read message	When the button “Close” is pressed it should close the read message page and show the email manager page.	It does close the page and shows the email manager page.	Pass
11	Label message	When the button is clicked in the email manager it should takes me to the label message page.	It does takes me to the label message page	Pass
12	Label message	When the button ‘Add Label’ is clicked it should add the label	The button does work when the button is clicked.	Pass
13	List Message	When I clicked the ‘List Messages’ button in the email manager it should list all the messages that has been added.	The button works when the button is clicked.	Pass

5. Screenshots of the Program working

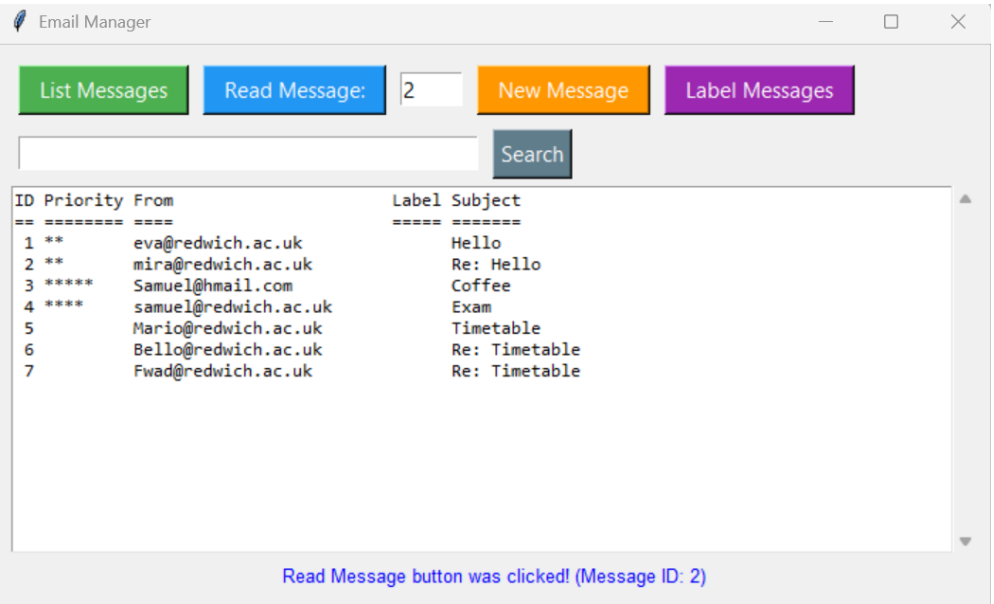
a. Screen 1 Email manager

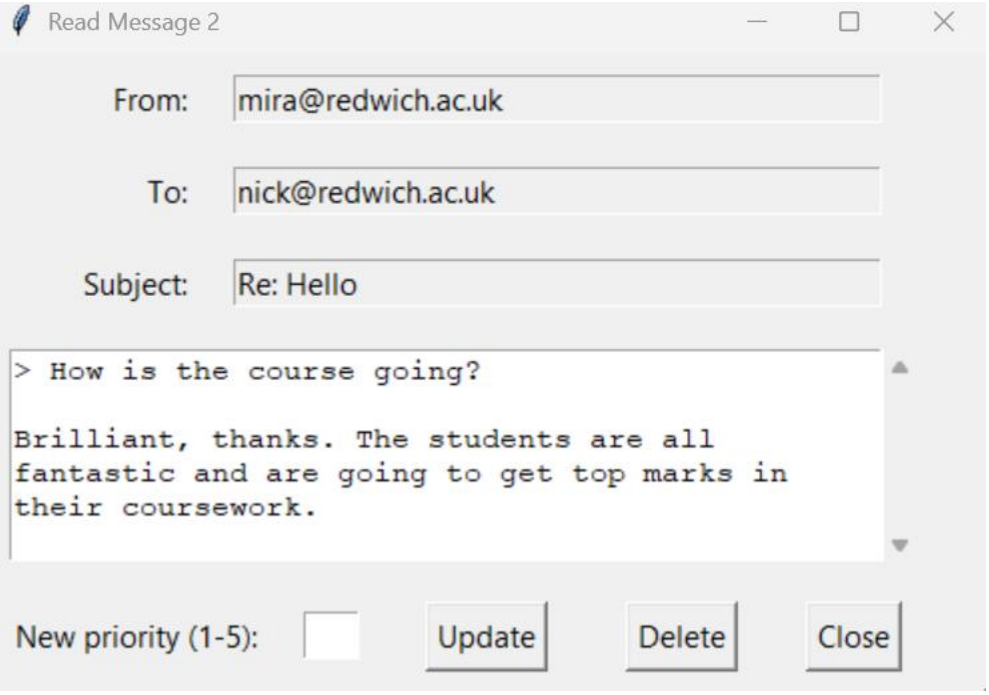


Brief description

This image shows Email manager Screen. Top of the line has buttons for different screens: List messages, read message, new message, and label message.

Screen 2- Read message.

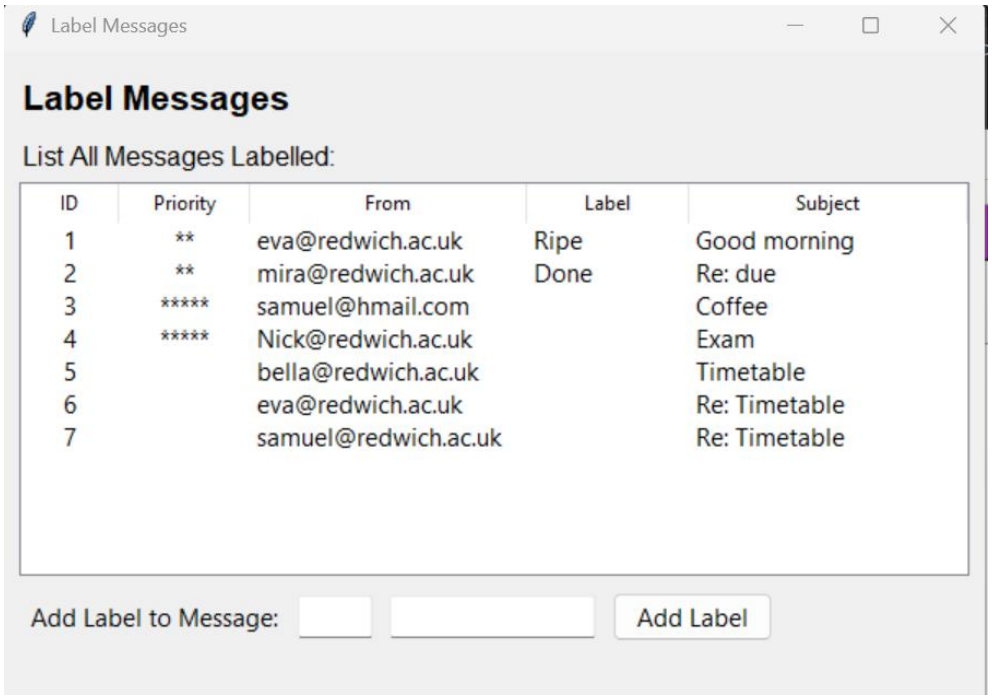




Brief description

When any ID was called it shows the ID information shows, and we can update, delete, and close the screen.

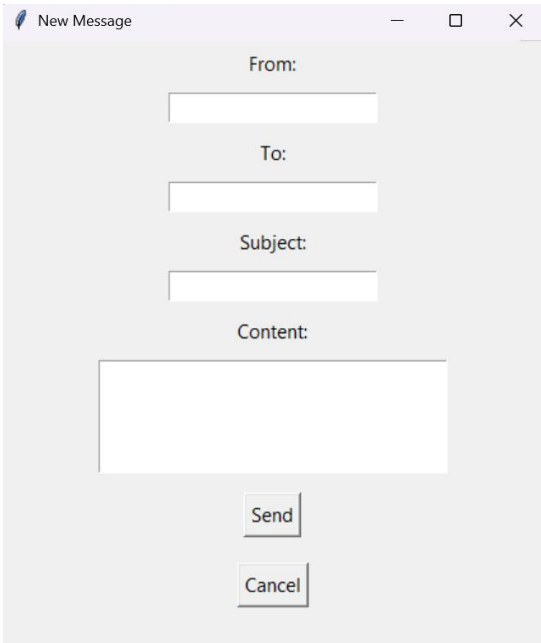
Screen 3 Label messages



Brief description

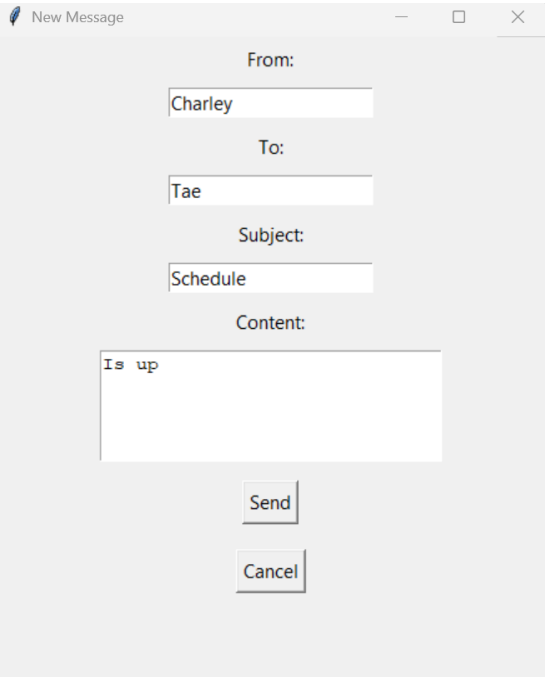
This screen is Label message we can newly add labels to add more ID.

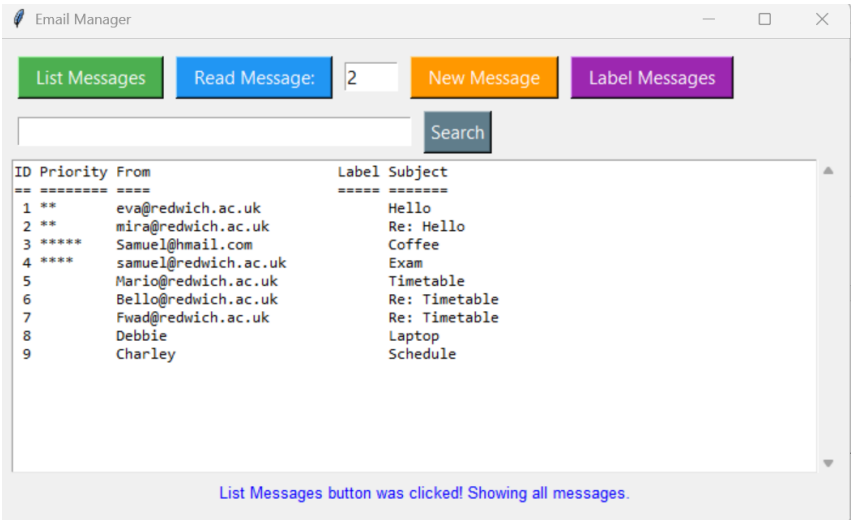
Screen 4 New messages



Brief description

I have typed a new message in the New Message window. When I click the button send it will transfer the message to the email manager.





This list shows the new message that as been add. Number 9 is the new added information.

Evaluation

This course work was hard for me at the beginning because I had minimum knowledge of doing coding. I did some tutorials, and I used W3 school to get used to doing code and learn the coding. I have watched YouTube videos to expend my knowledge more about the coding. After doing the research for the platform where I need to produce the coding, I choose VS code.

By doing this course work I have learned about the coding and how to work with new things which I had minimum knowledge about.

What went well?

Designing the wireframe and understanding about the coursework went well. The testing went well. These testing helped to identify any threats or any bugs that has been present in the code.

What went less well?

I faced challenging moment while doing this coursework. I tried to work on my list message to look it different, but I did not know how to do. I research about it how to make it work but some websites or W3 school information did not help much.

What I learned?

This coursework was challenging to do, but I was able to tackle and work with it smoothly with it. W3 school helped me with this work. I have learned how to work with python. I learned how prioritize my time and how to work with other assignment along with this. I have learned how to break down the difficult codding into smaller pieces to manage it more wisely.

AI declaration Form

UNIVERSITY OF GREENWICH

Declaration of AI Use

Please append this page to the end of your assignment when you have used AI during the process of undertaking the assignment to acknowledge the ways in which you have used it.

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – YES/NO
- To create an outline of the topic – YES/NO
- To explain concepts – YES/NO
- To support my use of language – YES/NO
- To summarise the following articles/resources:

1. Yes, I have used AI to Fix my 'new message' Code a bit.
2. Yes, I have used AI for understanding how to write the Evaluation
3. Yes, I have used AI for understanding what exactly to write in the test table.
4.
5.
6.

- In other ways, as described below: